

Shell Sort

希尔排序

本页面将简要介绍希尔排序。

定义

希尔排序（英语：Shell sort），也称为缩小增量排序法，是 [插入排序](#) 的一种改进版本。希尔排序以它的发明者希尔（英语：Donald Shell）命名。

过程

排序对不相邻的记录进行比较和移动：

1. 将待排序序列分为若干子序列（每个子序列的元素在原始数组中间距相同）；
2. 对这些子序列进行插入排序；
3. 减小每个子序列中元素之间的间距，重复上述过程直至间距减少为 。

性质

稳定性

希尔排序是一种不稳定的排序算法。

时间复杂度

希尔排序的最优时间复杂度为 。

希尔排序的平均时间复杂度和最坏时间复杂度与间距序列的选取有关。设间距序列为 ，下面给出 的两种经典选取方式，这两种选取方式均使得排序算法的复杂度降为 级别。

命题： 若间距序列为 （从大到小），则希尔排序算法的时间复杂度为 。

命题： 若间距序列为 （从大到小），则希尔排序算法的时间复杂度为 。

为证明这两个命题，我们先给出一个重要的定理并证明它，这个定理反应了希尔排序的最主要特征。

定理： 只要程序执行了一次 ，不管之后怎样调用 函数， 数组怎样变换，下列性质均会被一直保持：

证明：

我们先证明一个引理：

引理： 对于整数 、正整数 与两个数组 ，满足如下要求：

则我们将两个数组分别升序排序后，上述要求依然成立。

证明：

设数组 A 排序完为数组 B ，数组 B 排序完为数组 C 。

对于任何 i ， $A[i]$ 小等于数组 B 中的 i 个元素，也小等于数组 C 中的 i 个元素（这是因为 A 与 B 的元素可重集合是相同的）。

那么在可重集合 C 中，大等于 $A[i]$ 的元素个数不超过 i 个。

进而小于 $A[i]$ 的元素个数至少有 i 个，取出其中的 i 个，设它们为 S_i 。于是有：

所以 $A[i]$ 至少大等于 S_i 也即 C 中的 i 个元素，那么自然有 $A[i] \leq C[i]$ 。

证毕

再回到原命题的证明：

我们实际上只需要证明调用完 $sort$ 的紧接着下一次调用 $sort$ 后， A 的子列仍有序即可，之后容易用归纳法得出。下面只考虑下一个调用：

执行完 $sort$ 后，如下组已经完成排序：

而之后执行 $sort$ ，则会将如下组排序：

对于每个 i ，考虑如下两个组：

第二个组前面也加上“”的原因是可能 $A[i]$ 从而前面也有元素。

则第二个组就是引理 1 中的 A 数组，第一个组就是 B 数组， $B[i]$ 就是第二个组从 $A[i]$ 之后顶到末尾的长度， $B[i]$ 是第二个组中前面那个“”的长度， $B[i]$ 是第一个组去掉前 i 个后剩下的个数。

又因为有：

所以由引理 1 可得执行 $sort$ 将两个组分别排序后，这个关系依然满足，即依然有 $A[i] \leq B[i]$ 。

若有 $A[i] > B[i]$ ，容易发现取正整数 k 再加上若干个 k 即可得到 $A[i]$ ，则之前的情况已经蕴含了此情况的证明。

综合以上论述便有：执行完 $sort$ 依然有 $A[i] \leq B[i]$ 。

得证。

证毕

这个定理揭示了希尔排序在特定集合 A 下可以优化复杂度的关键，因为在整个过程中，它可以一致保持前面的成果不被摧毁（即 A 的子列分别有序），从而使后面的调用中，指针 i 的移动次数大大减少。

接下来我们单拎出来一个数论引理进行证明。这个定理在 OI 界因 [小凯的疑惑](#) 一题而大为出名。而在希尔排序复杂度的证明中，它也使得定理 1 得到了很大的扩展。

引理：若 a, b 均为正整数且互素，则不在集合 $\{ax + by \mid x, y \in \mathbb{N}\}$ 中的最大正整数为 $ab - a - b$ 。

证明：

分两步证明：

- 先证明方程 没有 均为非负整数的解：

若无非负整数的限制，容易得到两组解。

通过其通解形式，容易得到上面两组解是“相邻”的（因为）。

当 递增时， 递增， 递减，所以如果方程有非负整数解，必然会夹在这两组解中间，但这两组解“相邻”，中间没有别的解。

故不可能有非负整数解。 - 再证明对任意整数，方程 有非负整数解：

我们找一组解 满足（由通解的表达式，这可以做到）。

则有：

所以，又因为，所以，所以。

所以 为一组非负整数解。

综上得证。

证毕

而下面这个定理则揭示了引理 是如何扩展定理 的。

定理： 如果，则程序先执行完 与 后，执行 的时间复杂度为，且对于每个，其 的移动次数是 级别的。

证明：

对于 的部分， 的移动次数显然是 级别的。

故以下假设。

对于任意的正整数 满足，注意到：

又因为，故由引理，得存在非负整数，使得：。

即得：

由定理，得：

与

综合以上既有：。

所以对于任何，有。

在 Shell-Sort 伪代码中 指针每次减，减 次，即可使得，进而有，不满足 while 循环的条件退出。

证明完对于每个 的移动复杂度后，即可得到总的时间复杂度：

得证。

证毕

认真观察定理 的证明过程，可以发现：定理 1 可以进行“线性组合”，即以 为间隔有序，以 为间隔亦有序，则以 和 的非负系数线性组合仍是有序的。而这种“线性性”即是由引理 保证的。

有了这两个定理，我们可以命题 与 。

先证明命题：

证明：

将 写为序列的形式：

Shell-Sort 执行顺序为：.

分两部分去分析复杂度：

- 对于前面的若干个满足 的，显然有 的时间复杂度为 。

考虑对最接近 的项，有：

而对于 的，因为有，所以可得：

所以大等于 部分的总时间复杂度为：

- 对于后面剩下的满足 的项，前两项的复杂度还是，而对于后面的项，有定理 可得时间复杂度为：

再次利用 性质可得此部分总时间复杂度为（下式中 沿用了上一种情况中的含义）：

综上可得总时间复杂度即为 。

证毕

再证明命题：

证明：

注意到一个事实：如果已经执行过了 与，那么因为，所以由定理，每个元素只有与它相邻的前一个元素可能大于它，之前的元素全部都小于它。于是 指针只需要最多两次就可以退出 while 循环。也就是说，此时再执行，复杂度降为 。

更进一步：如果已经执行过了 与，我们考虑所有的下标为奇数的元素组成的子列与下标为偶数的元素组成的子列。则这相当于把这两个子列分别执行 与。那么也是一样，这时候再执行，相当于对两个子列分别执行，也只需要两个序列和的级别，即 的复杂度就可以将数组变为 间隔有序。

不断归纳，就可以得到：如果已经执行过了 与，则执行 的复杂度也只有 。

接下来分为两部分分析复杂度：

- 对于 的部分，则执行每个 的复杂度为 。

而 ，所以单词插入排序复杂度为 。

而这一部分元素个数是 级别的，所以这一部分时间复杂度为 。

- 对于 的部分，因为 ，所以这之前已经执行了 与 ，于是执行 的时间复杂度是 。

还是一样的，这一部分元素个数也是 级别的，所以这一部分时间复杂度为 。

综上可得总时间复杂度即为 。

证毕

空间复杂度

希尔排序的空间复杂度为 。

实现



C++¹Python

```
1 template <typename T>
2 void shell_sort(T array[], int length) {
3     int h = 1;
4     while (h < length / 3) {
5         h = 3 * h + 1;
6     }
7     while (h >= 1) {
8         for (int i = h; i < length; i++) {
9             for (int j = i; j >= h && array[j] < array[j - h]; j -= h) {
10                 std::swap(array[j], array[j - h]);
11             }
12         }
13         h = h / 3;
14     }
15 }
```

```
1 def shell_sort(array, length):
2     h = 1
3     while h < length / 3:
4         h = int(3 * h + 1)
5     while h >= 1:
6         for i in range(h, length):
7             j = i
8             while j >= h and array[j] < array[j - h]:
9                 array[j], array[j - h] = array[j - h], array[j]
10                j -= h
11         h = int(h / 3)
```

参考资料与注释

-
1. [希尔排序 - 维基百科，自由的百科全书](#) ↗
-

本页面最近更新：2023/10/4 21:50:08, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [NachtgeistW](#), [Alisahhh](#), [CroMarmot](#), [Enter-tainer](#), [iamtwz](#), [Menci](#), [partychicken](#), [Peanut-Tang](#), [shawlleyw](#), [ShwStone](#), [Tiphereth-A](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用